

Software Vulnerability Detection Using a Lightweight Graph Neural Network

Miles Farmer, Ekinan Ufuktepe, *Member, IEEE*, Anne Watson, Hialo Muniz Carvalho, Vadim Okun, *Member, IEEE*, Zineb Maasaoui, Kannappan Palaniappan, *Senior Member, IEEE*

Abstract—Large Language Models (LLMs) have emerged as a popular choice in vulnerability detection studies given their foundational capabilities, open source availability, and variety of models, but have limited scalability due to extensive compute requirements. Using the natural graph relational structure of code, we show that our proposed graph neural network (GNN) based deep learning model VulGNN for vulnerability detection can achieve performance almost on par with LLMs, but is 100 times smaller in size and fast to retrain and customize. We describe the VulGNN architecture, ablation studies on components, learning rates, and generalizability to different code datasets. As a lightweight model for vulnerability analysis, VulGNN is efficient and deployable at the edge as part of real-world software development pipelines.

Index Terms—software vulnerability detection, graph neural networks, deep learning, software security, large language models

I. INTRODUCTION

CONVENTIONAL techniques have shown limitations in detecting realistic and diverse software code vulnerabilities [1] [2] [3] [4], prompting researchers to pursue deep learning approaches. Systems based on deep learning provided notable improvements in detection performance compared to conventional methods. However, recent studies have also highlighted the limitations of deep learning-based vulnerability detectors when applied to complex, real-world data [5]. In such studies, benchmark GNN-based detectors lag behind LLM-based detectors in terms of accuracy and other detection metrics, though maintain a significant computational performance advantage due to their smaller model size. This performance advantage is of particular note and prompts further investigation, as one of the primary environments for which vulnerability detectors are desired, continuous integration/continuous delivery (CI/CD) pipelines, is often limited

in compute resources, only allocated enough to compile and test the software. LLMs of hundreds of millions or billions of parameters are not practical to host on such hardware, necessitating a more efficient solution.

Given the inherent underlying graph structure of code, it follows that deep models acting on graph-structured code data may provide improvements over models that act directly on source code. An LLM interpreting raw source code is forced to determine structural information internally through learned parameters, which may be inefficient in training and real-world inference. Meanwhile, a graph model offloads that responsibility to algorithmic preprocessing steps, thereby lowering the intensity of the model and potentially allowing for more efficient structure interpretation.

Despite this assumed advantage, recent studies still demonstrate a disparity in detection accuracy between GNNs and LLMs. This work investigated the continued feasibility of graph-based vulnerability detection approaches by implementing several key components of language models within a GNN architecture. Furthermore, we conducted several generalization studies to determine the real-world generalizability of our model, along with the impact of real-world versus synthetic data in the training process. Specifically, we investigated the following:

RQ1: How do GNN architecture variations affect vulnerability detection performance? We compare VulGNN to existing GNN architectures and investigate the impact of variations in the VulGNN architecture on vulnerability detection performance.

RQ2: How does VulGNN compare to LLMs in real-world vulnerability detection performance? We apply VulGNN to the DiverseVul [5] dataset and compare its detection performance to that of LLMs applied to the same data.

RQ3: Does VulGNN's capability generalize within and across datasets? We apply VulGNN to the “unseen projects” configuration of DiverseVul to test the model’s ability to generalize to entirely unseen codebases. Furthermore, we perform an “unseen dataset” experiment, wherein the model is trained on one dataset and tested on another from a different source, to ascertain cross-dataset generalizability.

RQ4: What is the impact of training on synthetic versus real-world data? A natural extension of the cross-dataset generalizability study, we perform a stepped ablation study to determine the impact of synthetic versus real-world training data on real-world vulnerability detection performance.

This paper makes the following main contributions:

Miles Farmer, Anne Watson, Ekinan Ufuktepe, and Kannappan Palaniappan are with the Department of Electrical Engineering and Computer Science at the University of Missouri, MO, USA. (e-mails: mlfcmg@missouri.edu, akw8kk@missouri.edu, euh46@missouri.edu, pal@missouri.edu)

Hialo Muniz Carvalho, Vadim Okun, and Zineb Maasaoui are with the Software and Systems Division at the National Institute of Standards and Technology, Gaithersburg, MD, USA (e-mails: hialo.munizcarvalho@nist.gov, vadim.okun@nist.gov, zineb.maasaoui@nist.gov)

Disclaimer: Any opinions, findings, and conclusions or recommendations expressed in this publication are those of the authors and do not necessarily reflect the views of the U.S. Government or agency thereof. Commercial products may be identified in this document, but such identification does not imply recommendation or endorsement by NIST, nor that the products identified are necessarily the best available for the purpose.

- 1) We propose an efficient GNN design and provide a systematic evaluation against LLM baselines.
- 2) We investigate the impact of key GNN architecture choices on vulnerability detection performance.
- 3) We analyze VulGNN’s vulnerability detection performance and generalizability by applying it to the DiverseVul [5] dataset and comparing its performance to that of LLMs.
- 4) We perform a stepped ablation study to compare the efficacy of real-world, synthetic, and mixed training data in real-world unseen vulnerability detection.
- 5) We publicly share our source code² for the model and its preprocessing steps for reproducibility.

The manuscript is organized as follows. In Section II, the background of different types of code graph representations is explained. We also mention the data representation approaches used in state-of-the-art graph and language-based detection models. Section III describes the model architecture and baseline parameters used in the experiments which follow. In Section IV, we compare our resulting model to existing approaches (addressing RQ2). In Section V, we interpret the meaning of the case study results and discuss their significance in the context of current approaches. Additionally, we note the potential limitations of the study. In Section VI, a summary of related work on deep learning-based vulnerability detection is given. Finally, Section VII concludes the paper and mentions the future work in relation to our study.

II. BACKGROUND AND FUNDAMENTALS

A. Code Graph Representations

This section provides some preliminary information on the types of code graph representations.

1) *Abstract Syntax Trees*: An Abstract Syntax Tree (AST) [6], [7] is a hierarchical structure representing the abstract syntactic structure of a source code. It captures the essential grammatical structure and semantics of a program, facilitating analysis, transformation, and optimization by abstracting away irrelevant details like comments. ASTs play a critical role in programming language processing and software development tools and techniques.

2) *Control-Flow Graphs*: A Control-Flow Graph (CFG) [8], [9] models the flow of control within a program, representing execution paths and decision points. Nodes represent basic blocks of codes, and edges indicate possible control flow. CFGs are key to program analysis, optimization, and error detection.

3) *Program Dependency Graphs*: A Program Dependency Graph (PDG) [10] represents relationships among program entities, capturing data and control dependencies. Data Dependency Graphs (DDGs) [11], [12] focus on the data flow, while Control Dependency Graphs (CDGs) [13], [14] focus on execution order. PDGs help in understanding information flow and control flow in a program.

4) *Code Property Graphs*: Code Property Graphs (CPGs) [15] provide a unified representation of code by integrating structural information from ASTs, CFGs, and PDGs. This holistic approach aids in advanced analysis, such as vulnerability detection and code comprehension. To generate CPGs we used Joern³, which has two versions of CPGs: CPG14 and CPG. CPG14 refers to schema version 1.4 of the code property graph, representing an earlier, stable layout of node and edge types used in Joern. Compared to newer CPG versions, CPG14 has fewer node types similar edge relationships, and limited support for modern language features or multi-language analysis. While CPG14 is ideal for legacy compatibility and reproducibility, the default (latest) CPG schema provides richer representations, more detailed semantic properties, and better support for evolving analysis workflows. This enhanced expressiveness and granularity in the modern CPG schema make it better fit for machine learning-based vulnerability detection, as it captures more contextual and semantic information that models can leverage for improved accuracy and generalization. Therefore, in this study, we have used the latest CPG version (not CPG14).

B. Sentence Embedding Models

In this study, we used tokenization to preprocess code language into streams of integers compatible with embedding. In particular, we used the StarCoder language model’s tokenizer, as it was trained on a variety of code in multiple modern languages, including C.

StarCoder is a GPT-style Transformer-decoder LLM proposed by Li et al. [16], built on top of the StarCoderBase model and Google AI PaLM 2 [17], which was trained using 80+ different programming languages with an emphasis on Python. The StarCoder and the StarCoderBase models were trained using a multi-query attention [18] architecture and Fill-in-the-Middle [19] objective. Its tokenizer is of a Byte Pair Encoding design.

III. DEEP LEARNING MODEL ARCHITECTURE AND PARAMETERS

The components of our VulGNN architecture are shown in Figure 1 and implemented using PyTorch and PyTorch Geometric [20]. Note that in this section and the paper overall, we use the terms graph neural network (GNN), graph convolutional network (GCN), and graph attention network (GAT) interchangeably. Since the VulGNN model described in this work is a GAT, the more general terms of GCN and GNN are applicable.

A. Overview

We propose VULGNN, a GNN designed for whole-graph binary classification of CPGs. The model uses attention-based message passing to capture dependencies between program elements and is configurable to operate with or without edge features. Nodes and edges are represented either as token

²<https://github.com/CIVA-Lab/VulGNN>

³<https://github.com/joernio/joern>

sequences or as discrete types. Token sequences are embedded into a shared latent space and enriched with sinusoidal positional encodings. The architecture is composed of stacked convolutional blocks with attention, followed by graph-level pooling and a binary classification head. In our source code, the hyperparameters are controlled through a unified configuration (`ModelConfig`), enabling systematic ablations.

B. Graph Representation

Let $G = (V, E)$ denote a graph with $|V| = N$ nodes and $|E|$ edges. Each node $i \in V$ provides either:

- a token sequence $X_i \in \mathbb{N}^{L_{\text{node}}}$ (default $L_{\text{node}} = 8$), or
- a discrete node type $t_i \in \{1, \dots, |\mathcal{T}|\}$, where $|\mathcal{T}| = 44$ is the number of node types in the CPG specification.

Similarly, each edge $(i, j) \in E$ provides either:

- a token sequence $A_{ij} \in \mathbb{N}^{L_{\text{edge}}}$ (default $L_{\text{edge}} = 16$), or
- a discrete edge type $r_{ij} \in \{1, \dots, |\mathcal{R}|\}$, where $|\mathcal{R}| = 20$ corresponds to the set of CPG edge relations.

A batch of graphs is represented with a standard batch vector as in PyTorch Geometric [20], and graph-level readout is performed via mean pooling.

C. Input Encoding

A single embedding matrix $E \in \mathbb{R}^{V \times d}$ (with $V = 49,152$, $d = 16$) is shared for node and edge tokens:

$$Z_i = E[X_i] \in \mathbb{R}^{L_{\text{node}} \times d}, \quad A_{ij}^{\text{emb}} = E[A_{ij}] \in \mathbb{R}^{L_{\text{edge}} \times d}.$$

Each token sequence is enriched with sinusoidal positional encodings [21] and flattened:

$$\tilde{Z}_i = \text{vec}(\text{PE}(Z_i)) \in \mathbb{R}^{L_{\text{node}} d},$$

$$\tilde{A}_{ij} = \text{vec}(\text{PE}(A_{ij}^{\text{emb}})) \in \mathbb{R}^{L_{\text{edge}} d}.$$

When using node or edge *types* in addition to or instead of tokenized sequences, we embed discrete IDs into low-dimensional vectors. In edge-type variants, edges are assigned embeddings $E_{\text{type}}[r_{ij}] \in \mathbb{R}^{d_e}$ with $d_e = 4$.

D. Convolutional Blocks

The backbone of VulGNN consists of stacked **ConvGroup** blocks. Each block applies:

- 1) **Message passing:** a `GeneralConv` [22] layer with dot-product attention, mean aggregation, and optional edge attributes. The `GeneralConv` is a general-purpose, highly-customizable graph convolutional layer inspired by You et al. [22] that features support for multiple aggregation schemes, additive or dot-product single- or multi-head attention, directed or undirected edges, and edge attributes. Its implementation in this work is configured with mean aggregation and dot-product attention.
- 2) **Nonlinearity:** PReLU activation.

- 3) **Normalization:** For normalization, we have used the `GraphNorm` [23] operation, where $\hat{h}_{i,j}$ is the j -th feature of the i -th node, is defined as

$$\text{GraphNorm}(\hat{h}_{i,j}) = \gamma_j \cdot \frac{\hat{h}_{i,j} - \alpha_j \cdot \mu_j}{\hat{\sigma}_j} + \beta_j \quad (1)$$

where $\mu_j = \frac{\sum_{i=1}^n \hat{h}_{i,j}}{n}$, $\hat{\sigma}_j^2 = \frac{\sum_{i=1}^n (\hat{h}_{i,j} - \alpha_j \cdot \mu_j)^2}{n} + \epsilon$; γ_j, β_j are learnable affine parameters; and α_j is a learnable weight parameter. The original implementation by Cai et al. [23] and the implementation we use from PyTorch Geometric [20] adds a small constant $\epsilon = 10^{-5}$ to the variance for numerical stability.

- 4) **Regularization:** Dropout with probability $p = 0.08$.

Formally, if $h^{(\ell)}$ denotes the node representation at layer ℓ , $h^{(\ell+1)} = \text{Dropout}\left(\text{GraphNorm}\left(\text{PReLU}\left(\mathcal{G}^{(\ell)}(h^{(\ell)}, E)\right)\right)\right)$,

where $\mathcal{G}^{(\ell)}$ is the `GeneralConv` operator.

The first block maps $L_{\text{node}} d \rightarrow D$ (default $8 \times 16 \rightarrow 128$), while subsequent blocks maintain hidden width $D = 128$. By default, the model stacks six such layers.

E. Graph Readout and Classification Head

After L convolutional layers, node embeddings are aggregated with global mean pooling:

$$\bar{h} = \frac{1}{N} \sum_{i=1}^N h_i^{(L)}. \quad (2)$$

A linear projection preserves hidden size D , followed by a final linear layer that outputs two logits for binary classification:

$$\hat{y} = W_2 \sigma(W_1 \bar{h}) \in \mathbb{R}^2. \quad (3)$$

F. Training Objective

The model is trained using a weighted binary cross-entropy with logits objective, to handle class imbalance in the training dataset. Given a raw logit $x \in \mathbb{R}$ class score and a binary target label $y \in \{0, 1\}$, and weight w , the per-sample binary cross-entropy loss is defined as

$$\ell(x, y) = -(w \cdot y \cdot \log(\sigma(x)) + (1 - y) \cdot \log(1 - \sigma(x))), \quad (4)$$

where $\sigma(\cdot)$ is the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (5)$$

To avoid numerical issues for large or small values of x , PyTorch uses an equivalent but stable formulation:

$$\ell(x, y) = \max(x, 0) - x \cdot y + \log(1 + e^{-|x|}). \quad (6)$$

$$w_k = \frac{N_{\text{samples}}}{N_{\text{classes}} \cdot N_k} \quad (7)$$

Where:

- w_k is the class weight for class k .
- N_{samples} is the total number of samples in the dataset.

- $N_{classes}$ is the total number of unique classes found in the data.
- N_k is the count of samples belonging to class k .

If a positive class weight $\text{pos_weight} \in \mathbb{R}^+$ is provided, the contribution of positive examples is scaled accordingly:

$$\ell(x, y) = \max(-x, 0) + (1 - y) \cdot x + (1 + (\text{pos_weight} - 1) \cdot y) \cdot (\log(1 + e^{-|x|})) \quad (8)$$

For a batch of N samples, the final loss is the mean over all examples:

$$L(x, y) = \frac{1}{N} \sum_{i=1}^N \ell(x_i, y_i). \quad (9)$$

The optimization is performed using Adaptive Moment (ADAM) estimation with learning rate 10^{-3} , decay rates beta1 (momentum) of 0.9, and beta2 (variance) of 0.999, batch size 400, and 25 epochs.

G. Model Variations

VulGNN supports configuration of several key aspects of the network, controlled by `ModelVariations`. The options for each key aspect are:

- **Node representation:** tokenized language nodes vs. discrete node types (both embedded).
- **Edge representation:** no edge features, embedded edge types, or embedded rich edge information.
- **Backbone:** homogeneous attention-based convolution (`GeneralConv`), **heterogeneous attention-based convolution (RGATConv)**.

This modularity enables systematic ablation of the effects of node features, edge features, and edge types on downstream performance.

Data processing was conducted across 6 to 13 servers, depending on the size of dataset and server availability, each of which has dual AMD EPYC 7713 CPUs (for a total of 128 cores) and 490GB of RAM. Model training was conducted on a single Nvidia H100 (80GB) running on a server with dual Xeon Platinum 8470s (total of 108 cores) and 2002 GB of RAM - though, we generally only utilize 32 cores for data loading and 96GB of RAM to keep the dataset in memory for low latency. Both of these steps can be accomplished on much less performant hardware, but this leads to longer processing and training times that inhibit efficient iterative testing. Inference can be accomplished on significantly less extensive hardware - the model uses only 449MB of VRAM with float32 used throughout. This could easily be lessened with little to no impact on classification performance with bfloat16 quantization [24], while potentially improving classification speed on newer hardware optimized for bfloat16 operations.

IV. CASE STUDY

To address RQ2 and RQ3, we compare the VulGNN system with existing state-of-the-art vulnerability detection systems. To accomplish this, we apply VulGNN to the DiverseVul dataset [5], allowing for direct performance comparisons to other detectors assessed in that work.

A. Datasets

We use the dataset provided by the DiverseVul [5] paper to benchmark VulGNN against existing methods. We refer to DiverseVul dataset, which includes Previous (deduplicated Devign [25], ReVeal [26], BigVul [27], CrossVul [28] and CVEFixes [29]) as described in [5], containing 523,956 functions, with 41,377 being vulnerable samples and the remaining 482,579 being non-vulnerable. The Software Assurance Reference Dataset (SARD/Juliet) created by NIST is a collection of programs with documented weaknesses in several different languages (C, C++, Java, PHP, C#), containing more than 450,000 test cases ranging from small pieces of code to production software [30] [31]. The Juliet Test Suite is a subset of the SARD/Juliet, containing 64,099 small synthetic test cases, originally created by the NSA's Center for Assured Software (CAS) specifically for use in testing static analysis tools. Our SARD/Juliet training set consists of 33,360 subset of cases with 12,303 vulnerable functions and 21,057 non-vulnerable functions.

B. Training and Test Procedures

Following the methodology of Chen et al. [5], we adopt the same dataset splitting strategy to ensure direct comparability. All experiments use deduplicated function-level samples, with an 80/10/10 ratio applied to construct the training, validation, and test splits, respectively. The validation set is used exclusively to select the best model checkpoint, i.e., the epoch yielding the highest validation F1-score. This checkpoint is then applied to the unseen test set for final performance reporting. This procedure minimizes overfitting and ensures fair evaluation on strictly held-out samples.

Given True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN), the performance metrics used in this paper are computed as follows:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (10)$$

$$Precision = \frac{TP}{TP + FP} \quad (11)$$

$$Recall = \frac{TP}{TP + FN} \quad (12)$$

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (13)$$

C. Test I: Randomly Chosen Sets

The first evaluation follows the random splitting protocol defined in DiverseVul. Here, the merged corpus (Previous + DiverseVul) is randomly partitioned into 80% training, 10% validation, and 10% test, while ensuring that no overlapping functions appear across splits. This approach provides a standard benchmark for assessing detector performance under balanced and representative data partitions. Importantly, this configuration permits direct comparison with prior baselines, as the exact same splitting criteria are maintained. Moreover, we use the exact splits provided by the DiverseVul paper to ensure our training, validation, and test sets are identical to those used previously.

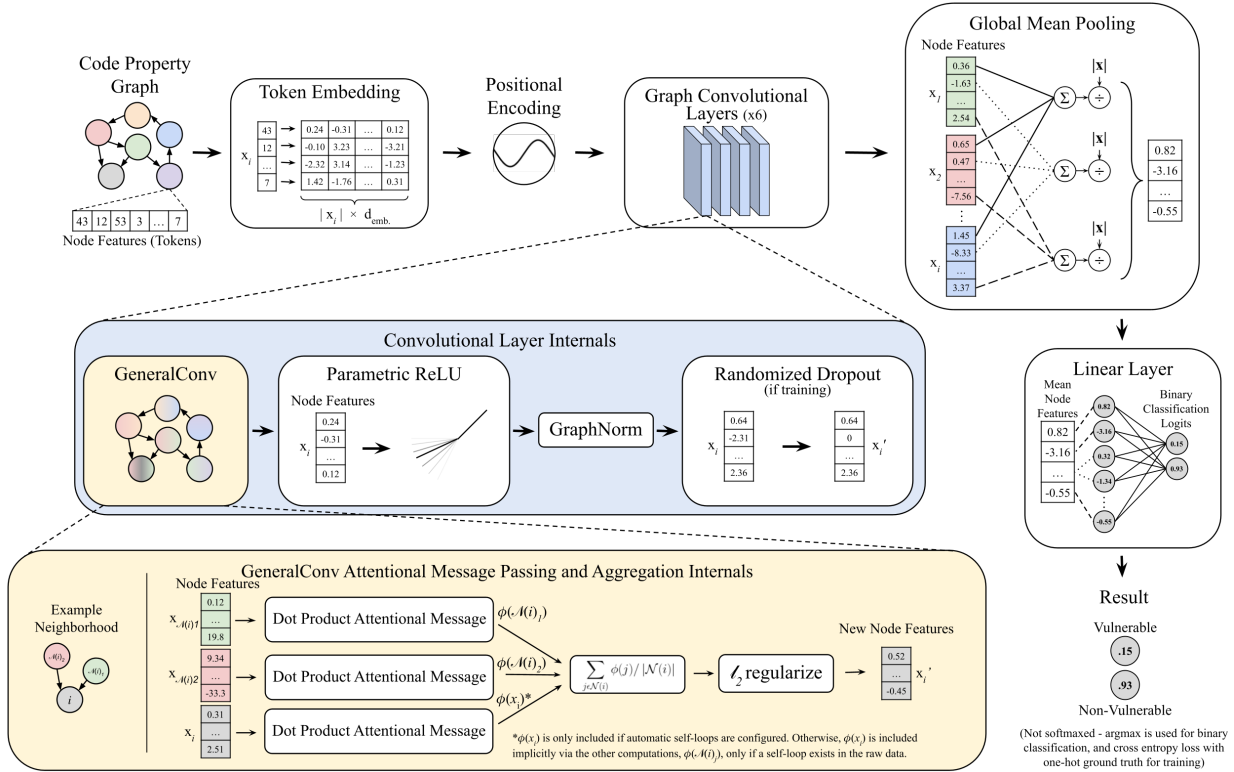


Fig. 1: VulGNN graph convolutional deep architecture with preprocessing (including language tokenization) to generate the input Code Property Graph (Row 1), the internal details of the graph neural network layers (Row 2), and the implementation of the GeneralConv operator with graph attention (Row 3). The output from the GNN after Global Mean Pooling is passed to a single linear classification layer. VulGNN does not include a hidden MLP layer after pooling.

D. Test II: Unseen Project Generalizability Test

Beyond random splits, we replicate the more realistic project-based evaluation of DiverseVul [5]. In this setting, entire projects are held out to construct the test set, ensuring that no functions from those projects appear in training or validation. Concretely, 95 projects are randomly chosen as unseen, while the remainder form the seen portion of the dataset. The seen subset is again divided into 90% training and 10% validation. This design probes a detector's ability to generalize across project boundaries, which is an essential capability for deployment in real-world CI/CD environments, where detectors must handle vulnerabilities in previously unseen codebases.

E. Training Data Augmentation Tests

To explore the impact of dataset composition, we also adapt the DiverseVul [5] ablation-style experiments. In particular, subsets of the training data are constructed to evaluate (i) the effect of scaling down to smaller training pools of only synthetic data, such as SARD/Juliet, and (ii) the balance between vulnerable and non-vulnerable samples. These controlled variations allow us to measure how performance shifts under constrained training regimes, highlighting the degree to which the detector benefits from larger and more diverse data sources.

1) *Test III: Real-World Training Data Ablation:* In Figure 2, to assess the impact of real-world training data on real-world detection performance, we perform a stepped ablation study, in which we train on a baseline of synthetic data with various amounts of added real-world data.

All configurations use the entirety of our Juliet dataset as training data. Then, we create seven training sets with varying amounts of real-world data added to this synthetic base. Such data is chosen at random from the DiverseVul training set, with the amount of data selected determined by the percent label. For example, the 0% scenario represents a training set consisting of the entirety of our Juliet dataset, but with 0% of the DiverseVul training set added. Meanwhile, 50% represents a training set consisting again of the entirety of our Juliet dataset, but with 50% of the DiverseVul training set also included.

We run the experiment on seven configurations: 0%, 10%, 20%, 40%, 60%, 80%, and 100% real world data included in the training set. We present the results in two formats: one that represents final performance on the DiverseVul test set after complete training, and another that shows the performance of each configuration on the DiverseVul validation set in each epoch.

2) *Test IV: Induced Training Data Balancing:* The DiverseVul dataset we use in all experiments is naturally highly class-imbalanced, with an approximate 1:11.66 vulnerable to non-vulnerable sample ratio. We use class weights to counteract

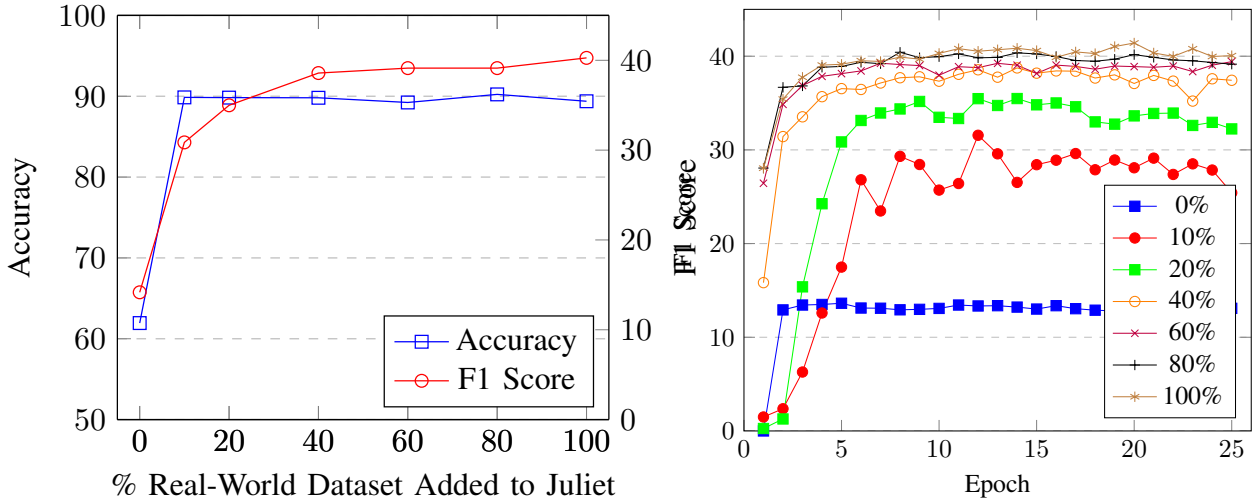


Fig. 2: Hybrid testing with different amounts of real-world data mixed with synthetic NIST Juliet Test Suite vulnerability data. VulGNN accuracy and F1-score (left graph) when trained on different percentages of real-world data randomly sampled from the training set of DiverseVul, combined with a baseline of synthetic Juliet Test Suite [32] which is a special subset of the NIST SARD/Juliet vulnerable code collection. That is, 0% is trained on a 33,360 sample subset of the Juliet dataset, while 100% is trained on the Juliet subset plus 419,164 training samples from DiverseVul. The right graph shows speed of training convergence using the DiverseVul validation set, and learning rate curves showing improvement with increasing percentage of real-world data.

this feature of the dataset, but this technique is not the only option to address the imbalance. In Figure 3 To examine the impact of this variable of the dataset on our wider results, we conduct an isolated test with induced rebalancing at various ratios. This is accomplished via random downsampling in the non-vulnerable training set, wherein some portion of the non-vulnerable training samples are randomly chosen and discarded such that the remaining non-vulnerable training samples form some desired ratio with the vulnerable training samples (e.g. 1:2). Following this procedure, we retain the imbalance in the test dataset, which is more representative of real-world data (only a minority of functions are vulnerable in reality), while still creating a more balanced training dataset. We continue to use class weights in all scenarios. It is already known that without this feature, any dataset imbalance greatly negatively impacts the F1 score.

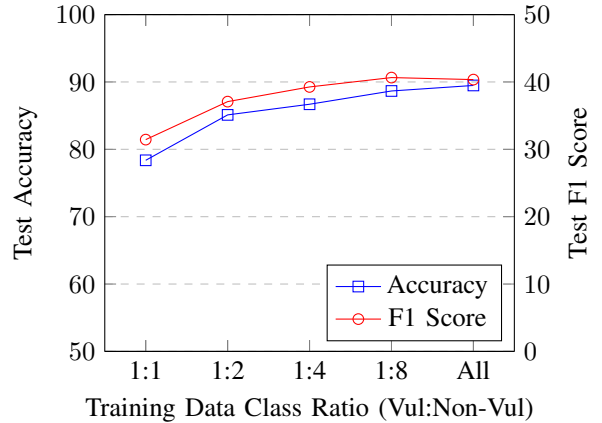


Fig. 3: Accuracy and F1 Score test results on different Vul:Non-Vul training ratios

F. Results

The results of tests *I* and *II* are shown in Table I. We include the performance data of models tested in the DiverseVul paper for comparison convenience. When sorting by F1-score, VulGNN leads any other detector by about 6% in unseen project performance, achieving this improvement with a minor expense to accuracy.

It is also important to interpret these results in the context of model size. Table II shows the number of parameters in each model. Note that graph-based detectors also require significant preprocessing steps to generate graph data from source code, so the number of parameters cannot be used as a direct efficiency comparison. However, the graphs used by VulGNN are often generated as a byproduct of other development tools

that conduct static analysis, so this overhead may be negligible in real world projects where these artifacts can be reused.

V. DISCUSSION

A. Discussion of Results

Our work culminates in a detection system that, per our test findings, is competitive with state-of-the-art systems in both unseen and in-distribution evaluations. In the Unseen Project configuration of DiverseVul, VulGNN improves F1 by approximately 6% compared to the best baseline, while maintaining competitive accuracy. Although LLM-based models achieve higher raw accuracy in some settings, they often suffer from poor recall and huge parameter counts.

TABLE I: Vulnerability detection performance of deep learning models for two scenarios. The best performing result in each column is shown in bold. Results in the “*Train on Prev+Diverse, Test on Unseen Projects*” column represent results from the experiment described in Section IV-D. Results in the “*Train & Test on Prev+Diverse*” column represent results from the experiment described in Section IV-C. All results, except for VulGNN, are from DiverseVul [5] (see their Tables 4 and 5) (test on same and unseen data respectively; marked with *), and Table 6 (optimized hyperparameters; marked with †). See the aforementioned sections for further details on dataset split and experiment setup.

Family	Architecture	Train on Prev+Diverse, Test on Unseen Projects					Train (80%) & Test (10%) on Prev+Diverse (no overlap)				
		F1	Acc	Prec	Rec	FPR	F1	Acc	Prec	Rec	FPR
GNN	ReVeal*	8.67	85.88	5.67	18.46	11.58	29.76	82.81	23.75	39.83	12.87
	VulGNN (Ours)	18.17	93.17	16.18	20.72	4.08	40.35	89.48	36.84	44.6	6.63
BERT	RoBERTa*	4.40	95.59	10.46	2.78	0.90	34.98	91.68	46.02	28.22	2.85
	CodeBERT*	11.94	94.19	13.34	10.80	2.65	37.85	90.48	39.25	36.54	4.87
	CodeBERT [†]	14.74	92.16	12.21	18.60	5.06	41.72	89.39	36.97	47.89	7.04
	GraphCodeBERT*	9.25	94.74	12.48	7.35	1.95	36.79	90.32	38.18	35.51	4.96
GPT-2	GPT-2 Base*	6.02	95.06	9.82	4.34	1.51	33.03	91.73	46.18	25.71	2.58
	CodeGPT*	7.72	94.47	9.86	6.35	2.19	35.23	91.36	43.48	29.62	3.32
	PolyCoder*	11.39	92.73	10.25	12.81	4.24	31.96	91.97	48.76	23.78	2.15
	PolyCoder [†]	13.63	89.76	9.84	22.16	7.68	36.68	86.48	29.19	49.36	10.32
T5	T5 Base*	9.73	96.16	34.00	5.68	0.42	42.33	91.96	49.14	37.17	3.32
	CodeT5 Small*	9.39	94.91	13.35	7.24	1.78	45.10	91.85	48.41	42.22	3.88
	CodeT5 Small [†]	17.21	93.87	16.95	17.48	3.24	48.28	89.57	39.80	61.33	7.99
	CodeT5 Base*	9.14	95.56	18.03	6.12	1.05	45.69	92.11	50.36	41.81	3.55
	NatGen*	9.30	95.49	17.38	6.35	1.14	47.15	92.30	51.81	43.25	3.47

TABLE II: The number of learnable weight parameters for each network.

Family	Architecture	Number of Parameters
GNN	ReVeal	1.28 Million
	VulGNN	1.10 Million
BERT	RoBERTa	125 Million
	CodeBERT	125 Million
	GraphCodeBERT	125 Million
GPT-2	GPT-2 Base	117 Million
	CodeGPT	124 Million
	PolyCoder	160 Million
T5	T5 Base	220 Million
	CodeT5 Small	60 Million
	CodeT5 Base	220 Million
	NatGen	220 Million

In terms of efficiency, VulGNN has only 1.1M parameters compared to 60M–220M in LLMs. This is two orders-of-magnitude reduction in memory footprint and compute cost. Even without precise inference benchmarks, it is evident that VulGNN can be deployed on commodity GPUs and potentially CPUs within CI/CD pipelines, while LLMs require dedicated high-memory GPUs. Thus, VulGNN balances detection capability with practical deployability.

B. Threats to Validity

1) *Internal Threats to Validity*: Internal validity refers to whether the observed outcomes in our study are genuinely caused by the factors under investigation rather than hidden confounders. To reduce such risks, we fully automated the data preparation, preprocessing, and evaluation pipelines, ensuring consistent treatment across all experimental runs.

However, threats remain. In particular, parameter configurations and implementation choices (e.g., preprocessing heuristics, splitting strategies, or default hyperparameters of tools) may have inadvertently influenced the results. Another potential source of bias lies in the selection of subsets of the dataset for evaluation, which could favor certain vulnerability types over others. We mitigated this by using multiple runs, sampling randomization, and validation on independent splits, but confounding influences cannot be entirely ruled out.

2) *External Threats to Validity*: External validity concerns the degree to which our results generalize beyond the studied context. While we relied on DiverseVul, which is one of the largest and most carefully curated datasets currently available, the findings may not directly generalize to all software domains or to industrial projects. Real-world code bases often differ in scale, coding style, testing practices, and developer workflows, and such differences can affect how vulnerabilities manifest and how models perform in practice.

Moreover, vulnerability datasets continue to evolve. Even though DiverseVul contains a larger number of instances with higher label correctness (validated by random sampling) than previous datasets, it is still subject to inherent limitations of any benchmark dataset. For instance, it may not fully represent emerging vulnerability types, non-C/C++ projects, or industrial code developed under proprietary processes. Replications in additional settings, particularly on industrial datasets or longitudinal corpora, would be valuable to assess the generalizability of our findings.

To further mitigate threats of project-specific bias, we conducted two complementary evaluation setups: (1) training on Prev+Diverse and testing on entirely unseen projects, and (2) training and testing on Prev+Diverse with no overlaps. The unseen-project setting provides stronger evidence of generalizability, as models are not exposed to project-specific coding styles or artifacts during training. While this design reduces the risk of overfitting to particular projects, it still does not guarantee transferability to industrial contexts, where code bases may differ substantially in complexity, scale, or development practices.

3) *Construct Threats to Validity*: Construct validity relates to whether the measures and datasets used in the study adequately capture the theoretical concepts of interest. Our study relied on publicly available vulnerability datasets. However, prior work (including DiverseVul itself) has highlighted significant limitations in these resources: for example, incorrect labels, incomplete mappings of vulnerabilities that span across multiple functions, and inconsistencies across datasets such as BigVul [27], CVEFixes [29], and CrossVul [28]. By adopting DiverseVul, we aimed to mitigate these issues, since it provides more data and a higher rate of correct labeling (based on manual inspection of a random sample).

At the same time, our decision not to rely exclusively on synthetic datasets such as SARD/Juliet introduces another trade-off. While SARD/Juliet provides more reliably correct labels, its data consists of small, often artificial code samples that are less representative of real-world software, limiting ecological validity. Thus, while our dataset choice strengthens the realism of the study, there remains a residual threat that the chosen benchmarks may still not perfectly capture the construct of “real-world vulnerability detection.”

4) *Conclusion Threats to Validity*: Conclusion validity concerns whether the conclusions we draw from the observed results are justified given the evaluation measures. Our analysis relied on commonly used performance metrics (precision, recall, F1-score, accuracy, and false positive rate), which provide complementary views of model behavior. Using multiple measures reduces the risk of drawing misleading conclusions

based on a single metric (e.g., high accuracy in imbalanced data).

However, threats remain. First, while these metrics quantify performance, they may not fully reflect the practical utility of a vulnerability detection system in real-world development. For instance, a low false positive rate may still translate into a large number of alerts in large projects, potentially reducing tool adoption despite favorable metrics. Second, results may be sensitive to dataset imbalance: metrics such as accuracy can be inflated when non-vulnerable instances dominate, while F1-score may emphasize trade-offs between precision and recall but overlook costs associated with misclassifications. Finally, since we did not perform statistical tests, we cannot formally assess whether observed differences between models are significant or due to chance.

To mitigate the limitations in metric representation, we reported multiple metrics to provide a holistic perspective, and we interpreted results cautiously, emphasizing observed patterns and relative performance rather than absolute claims of superiority. To minimize the impact of random data sampling and parameter initialization, every configuration in each experiment is trained and tested three times, with the presented test results being the average over the three runs. Nonetheless, we acknowledge that further studies with larger samples, alternative evaluation frameworks, or statistical testing would be needed to strengthen the robustness of our conclusions.

Another consideration is that results varied across the two evaluation designs. The Train on Prev+Diverse, Test on Unseen Projects setting provides stronger evidence of generalizability, as the model is not exposed to project-specific artifacts during training, while the Train and Test on Prev+Diverse (with no overlaps) setting reflects overall performance on the combined dataset. This difference highlights that conclusions must be interpreted within the context of the evaluation setup, and that replications with additional testing scenarios would further strengthen confidence in the findings.

VI. RELATED WORK

One of the first GNN-based methods to detect code vulnerabilities was presented by Zhou et. al [25], which was introduced as *Devign* that learned from AST, CFG, and DFG code graph features to detect code vulnerabilities on open-source C projects. Cheng et al. [42] used GNNs to embed code fragments, preserving control-flow, data-flow, and natural language information in their *XFG* representation (program slices). Unlike VulGNN, Cheng et al. [42] used program slicing on PDGs, which is a code graph representation that is commonly used to only contain relevant code entities (nodes and edges) that are related to the vulnerability. Their GNN is based on two graph convolutional layers, while we used a different convolutional layer called GeneralConv with five layers. In addition, Cheng et al. [42] worked on a semi-synthetic and trained on only 10 CWE classes, while we have trained on semi-synthetic and real-world vulnerabilities that included 150 CWE classes. Furthermore, they used Doc2Vec’s tokenizer [43] to embed their code, while we used StarCoder.

Some other early GNN-based methods demonstrated the benefit of modeling code structure, but often struggled with

TABLE III: Comparison of VulGNN and Representative recent GNN-Based Vulnerability Detectors.

Model	Year	Graph / Input Representation	Model Architecture / Key Innovation	Languages / Scope	Evaluation Dataset(s)	Training Code Availability
VulGNN (Ours)	2025	CPG / AST+CFG+PDG	GNN design emphasizing generalization, regularization, cross-project robustness	C/C++	SARD/Juliet + DiverseVul (Real-world)	✓
ReVeal [26]	2022	CPG	Gated Graph Neural Network (GGNN) with SMOTE and triplet loss	C/C++	Devign [25] + vulnerable samples collected from Chrome and Debian projects	✓
BGNN4VD [33]	2021	Code graph (bidirectional edges)	Bidirectional GNN integrating forward	backward flows	C/C++ & NVD / GitHub projects	✗
ReGVD [34]	2022	Token-level graph over code tokens	Lightweight GNN with residual layers, mixed pooling	Multi-language (incl. C/C++)	CodeXGLUE vulnerability benchmark	✓
LineVD [35]	2022	PDG at statement granularity	Hybrid Transformer + GAT — statement-level classification	C/C++	Real vulnerability projects (Linux, Chromium)	✓
MVD [36]	2022	Flow-sensitive PDG (statement-level)	Flow-sensitive GNN (FS-GNN) tailored for memory errors	C/C++(memory bugs)	Memory-related vulnerabilities (4,353 samples)	✗
TACSan [37]	2024	Graph over TAC (3-address code)	GNN over normalized intermediate representation	C/C++	SARD + multiple CWE datasets	✗
Joint Graph + Transformer [38]	2025	Merged multi-graph (AST, CFG, PDG, etc.)	GAT + Transformer fusion, pre/post fusion	C/C++	SARD + “Real-Vul” dataset	✗
ExplainVulD [39]	2025	CPG (AST+CFG+DFG)	Edge-aware GATv2 + dual-channel embeddings + explanation module	C/C++	ReVeal dataset	✗
DSHGT [40]	2024	Heterogeneous CPG (typed nodes/edges)	Heterogeneous graph transformer, dual-supervisor architecture	C/C++ (and cross-language)	Real-world projects + transfer tests	✗
VulTriNet [41]	2025	Intermediate representation (IR) + code graph	GNN on IR + noise-mitigation	C/C++	(not fully detailed)	✗

scalability and generalization. BGNN4VD [33] improved over static analyzers by using bidirectional edges, while ReGVD [34] showed that even lightweight token-level GNNs could achieve competitive results on CodeXGLUE [44]. However, both approaches were evaluated mainly within single datasets, making their robustness under cross-project settings less clear. In contrast, VulGNN explicitly targets generalization across large-scale, real-world C/C++ projects, prioritizing stability under domain shifts rather than only within-dataset performance.

Later works introduced more sophisticated graph representations or hybrid architectures. LineVD [35] and MVD [36] refined statement-level program dependence graphs for detecting memory vulnerabilities, and TACSan [37] normalized code into intermediate representations to reduce noise. While effective, these models were tailored to specific vulnerability types or relied heavily on curated datasets. By comparison, VulGNN is designed as a general-purpose detector, maintaining consistent performance across varied vulnerability classes and projects without overfitting to one representation or benchmark.

More recent advances emphasize either performance boosts through model complexity or explainability. For instance, Joint Graph + Transformer [38] and DSHGT [40] integrate transformers or heterogeneous graph modeling to achieve higher accuracy, while ExplainVulD [39] and VulPathFinder [45]

provide interpretability by surfacing influential nodes or paths. Other works such as VulTriNet [41] continue refining graph construction and noise handling. These systems highlight promising directions but often involve heavier models or focus narrowly on explanation. VulGNN, by contrast, strikes a balance between expressiveness and efficiency, aiming for reproducible, cross-project robustness with lower complexity, which makes it more deployable in practical vulnerability detection pipelines.

Finally, in Table III we show the differences between our proposed model and other GNN models. We did not directly benchmark VulGNN against every prior GNN-based vulnerability detector included in the related work due to significant reproducibility and comparability limitations. Among the cited studies, only two (ReGVD [34] and LineVD [35]) provide publicly available training code, making them the only reproducible GNN baselines identified. However, both rely on large pre-trained transformer backbones (e.g., GraphCodeBERT and CodeBERT) exceeding 125 million learnable parameters, resulting in models that are orders of magnitude larger and computationally heavier than VulGNN. Additionally, the LineVD [35] is a statement-level vulnerability detection model (node classification), while VulGNN is a function-level vulnerability detection (graph classification) model. In other words, our model and LineVD approach the vulnerability detection problem at different granularities

of code, which makes them unfit to compare. In contrast, our model is intentionally designed to be lightweight and efficient while maintaining competitive predictive capability. Furthermore, many other GNN-based detectors employ different code representations (e.g., program dependence graphs or intermediate representations), target languages other than C/C++, or address tasks such as statement-level or multi-class vulnerability classification, which are conditions incompatible with our function-level binary classification setup. Given these disparities, integrating those approaches into our experimental framework would require non-trivial re-engineering and could lead to unfair or misleading comparisons. Consequently, our evaluation prioritizes generalization, reproducibility, and deployment feasibility, which are key goals that distinguish VulGNN from previous GNN architectures. Table III already highlights the architectural and dataset-level differences across these studies, underscoring that our decision reflects a deliberate design focus rather than an omission of comparable baselines.

VII. CONCLUSIONS AND FUTURE WORK

VulGNN is a lightweight graph neural network based deep architecture for code vulnerability detection that outperforms the state-of-the-art ReVeal GNN and is comparable to large language models, but with two orders of magnitude fewer weights. We show that incorporating even a small amount of real-world vulnerable code examples ($\sim 42\text{K}$ samples or 10% of DiverseVul) to balance the purely synthetic NIST Juliet dataset, significantly improves accuracy from 62% to 90%. F1 performance continues improving from 31% to almost 40% as the amount of real-world DiverseVul samples increases from 10% to 40% ($\sim 167.7\text{K}$ DiverseVul samples). The lightweight nature of VulGNN makes it significantly more practical to include in code development environments, such as continuous integration (CI) pipelines, and could prove a powerful tool in combination with emerging AI code reviewers.

Future work primarily involves examining other model variations, training techniques, and generalization across programming languages. Variations of particular interest include using different convolutional layers, using a pre-trained “foundation” embedding model to create node representations, and introducing data augmentation for enhanced network communicativity.

VIII. ACKNOWLEDGMENTS

This work was partially supported by the MizzouForward Undergraduate Research Training Grant.

This work was partially supported by the U.S. National Science Foundation award 2243619 and Army Engineer Research and Development Center - Information Technology Laboratory (ERDC-ITL) contract W912HZ23C0041.

REFERENCES

- [1] S. M. Ghaffarian and H. R. Shahriari, “Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey,” *ACM Computing Surveys*, vol. 50, no. 4, pp. 1–36, 2017.
- [2] P. Zeng, G. Lin, L. Pan, Y. Tai, and J. Zhang, “Software vulnerability analysis and discovery using deep learning techniques: A survey,” *IEEE Access*, vol. 8, pp. 197 158–197 172, 2020.
- [3] H. Hanif, M. H. N. M. Nasir, M. F. Ab Razak, A. Firdaus, and N. B. Anuar, “The rise of software vulnerability: Taxonomy of software vulnerabilities detection and machine learning approaches,” *Journal of Network and Computer Applications*, vol. 179, p. 103009, 2021.
- [4] B. Shuai, H. Li, L. Zhang, Q. Zhang, and C. Tang, “Software vulnerability detection based on code coverage and test cost,” in *11th Int. Conf. on Computational Intelligence and Security*, 2015, pp. 317–321.
- [5] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, “DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection,” in *Proc. of Int. Symp. on Research in Attacks, Intrusions and Defenses (RAID)*, 2023, pp. 654–668.
- [6] P. J. Landin, “The next 700 programming languages,” *Communications of the ACM*, vol. 9, no. 3, pp. 157–166, 1966.
- [7] P. Lucas, “Formal semantics of programming languages: Vdl,” *IBM Journal of Research and Development*, vol. 25, no. 5, pp. 549–561, 1981.
- [8] F. E. Allen, “Control flow analysis,” *ACM Sigplan Notices*, vol. 5, no. 7, pp. 1–19, 1970.
- [9] R. T. Prosser, “Applications of boolean matrices to the analysis of flow diagrams,” in *Papers presented at the December 1-3, 1959, eastern joint IRE-AIEE-ACM computer conference*, 1959, pp. 133–138.
- [10] J. Ferrante, K. J. Ottenstein, and J. D. Warren, “The program dependence graph and its use in optimization,” *ACM Transactions on Programming Languages and Systems*, vol. 9, no. 3, pp. 319–349, 1987.
- [11] D. J. Kuck, R. H. Kuhn, D. A. Padua, B. Leasure, and M. Wolfe, “Dependence graphs and compiler optimizations,” in *Proceedings of the 8th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1981, pp. 207–218.
- [12] K. J. Ottenstein, *Data-flow graphs as an intermediate program form*. Purdue University, 1978.
- [13] J. Ferrante and K. J. Ottenstein, “A program form based on data dependency in predicate regions,” in *Proceedings of the 10th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, 1983, pp. 217–236.
- [14] I. Natour, “On the control dependence in the program dependence graph,” in *Proceedings of the 1988 ACM sixteenth annual conference on Computer science*, 1988, pp. 510–519.
- [15] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and discovering vulnerabilities with code property graphs,” in *IEEE Symp. on Security and Privacy*, 2014, pp. 590–604.
- [16] R. Li, L. B. allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, J. Lamy-Poirier, J. Monteiro, N. Gontier, M.-H. Yee, L. K. Umapathi, J. Zhu, B. Liptkin, M. Oblukov, Z. Wang, R. Murthy, J. T. Stillerman, S. S. Patel, D. Abulkhanov, M. Zocca, M. Dey, Z. Zhang, U. Bhattacharyya, W. Yu, S. Luccioni, P. Villegas, F. Zhdanov, T. Lee, N. Timor, J. Ding, C. S. Schlesinger, H. Schoelkopf, J. Ebert, T. Dao, M. Mishra, A. Gu, C. J. Anderson, B. Dolan-Gavitt, D. Contractor, S. Reddy, D. Fried, D. Bahdanau, Y. Jernite, C. M. Ferrandis, S. Hughes, T. Wolf, A. Guha, L. V. Werra, and H. de Vries, “StarCoder: May the source be with you!” *Transactions on Machine Learning Research*, 2023.
- [17] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel, “PaLM: Scaling language modeling with pathways,” *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.
- [18] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” *arXiv preprint arXiv:1911.02150*, 2019.
- [19] M. Bavarian, H. Jun, N. Tezak, J. Schulman, C. McLeavey, J. Tworek, and M. Chen, “Efficient training of language models to fill in the middle,” *arXiv preprint arXiv:2207.14255*, 2022.
- [20] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.

- [22] J. You, Z. Ying, and J. Leskovec, "Design space for graph neural networks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, 2020, pp. 17 009–17 021.
- [23] T. Cai, S. Luo, K. Xu, D. He, T.-Y. Liu, and L. Wang, "GraphNorm: A principled approach to accelerating graph neural network training," in *Proc. 38th Int. Conf. on Machine Learning*, vol. 139, 2021, pp. 1204–1215.
- [24] D. D. Kalamkar, D. Mudigere, N. Mellempudi, *et al.*, "A study of bfloat16 for deep learning training," *CoRR*, vol. abs/1905.12322, 2019. [Online]. Available: <http://arxiv.org/abs/1905.12322>
- [25] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [26] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2022.
- [27] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ code vulnerability dataset with code changes and CVE summaries," in *Proc. 17th Int. Conf. on Mining Software Repositories*, 2020, p. 508–512.
- [28] G. Nikitopoulos, K. Dritsa, P. Louridas, and D. Mitropoulos, "CrossVul: A cross-language vulnerability dataset with commit data," in *Proc. 29th ACM Joint Meeting on European Software Engineering Conf. and Symp. on the Foundations of Software Engineering*, 2021, p. 1565–1569.
- [29] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software," in *ACM Proc. 17th Int. Conf. on Predictive Models and Data Analytics in Software Engineering*, 2021, p. 30–39.
- [30] P. E. Black, "A software assurance reference dataset: Thousands of programs with known bugs," *J. Research National Institute of Standards and Technology*, vol. 123, Apr. 2018. [Online]. Available: <https://doi.org/10.6028/jres.123.005>
- [31] NIST. Software assurance reference dataset. [Online]. Available: <https://samate.nist.gov/SARD/>
- [32] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "VulCNN: An image-inspired scalable vulnerability detection system," *IEEE/ACM 44th Int. Conf. on Software Engineering (ICSE)*, pp. 2365–2376, 2022.
- [33] S. Cao, X. Sun, L. Bo, Y. Wei, and B. Li, "Bgnn4vd: Constructing bidirectional graph neural-network for vulnerability detection," *Information and Software Technology*, vol. 136, p. 106576, 2021.
- [34] V.-A. Nguyen, D. Q. Nguyen, V. Nguyen, T. Le, Q. H. Tran, and D. Phung, "ReGVD: Revisiting graph neural networks for vulnerability detection," in *Proc. ACM/IEEE 44th Int. Conf. on Software Engineering: Companion Proc.*, 2022, pp. 178–182.
- [35] D. Hin, A. Kan, H. Chen, and M. A. Babar, "Linevd: Statement-level vulnerability detection using graph neural networks," in *Proceedings of the 19th international conference on mining software repositories*, 2022, pp. 596–607.
- [36] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, and C. Tao, "MVD: Memory-related vulnerability detection based on flow-sensitive graph neural networks," in *Proc. 44th Int. Conf. on Software Engineering*, 2022, pp. 1456–1468.
- [37] Q. Zeng, D. Xiong, Z. Wu, K. Qian, Y. Wang, and Y. Su, "Tacsan: Enhancing vulnerability detection with graph neural network," *Electronics*, vol. 13, no. 19, p. 3813, 2024.
- [38] D. Jin, C. He, Q. Zou, Y. Qin, and B. Wang, "Source code vulnerability detection based on joint graph and multimodal feature fusion," *Electronics*, vol. 14, no. 5, p. 975, 2025.
- [39] R. Haque, A. Ali, S. McClean, and N. Khan, "Explainable vulnerability detection in c/c++ using edge-aware graph attention networks," *arXiv preprint arXiv:2507.16540*, 2025.
- [40] T. Zhang, R. Xu, J. Zhang, Y. Liu, X. Chen, J. Yin, and X. Zheng, "Dshgt: Dual-supervisors heterogeneous graph transformer—a pioneer study of using heterogeneous graph learning for detecting software vulnerabilities," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–31, 2024.
- [41] Y. Yang, Y. Yao, X. Lv, and W. Chen, "Vultrinet: A software vulnerability detection method based on tri-channel network," *Information and Software Technology*, p. 107893, 2025.
- [42] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "DeepWukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Transactions on Software Engineering and Methodology*, vol. 30, no. 3, pp. 1–33, 2021.
- [43] Q. Le and T. Mikolov, "Distributed representations of sentences and documents," in *Int. Conf. on Machine Learning*, 2014, pp. 1188–1196.
- [44] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, L. Shou,

L. Zhou, M. Tufano, M. Gong, M. Zhou, N. Duan, N. Sundaresan, S. K. Deng, S. Fu, and S. Liu, "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in *Conf. on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

- [45] N. Atashin, B. T. Ladani, and M. Sharbaf, "Learning to locate: Gnn-powered vulnerability path discovery in open source code," *arXiv preprint arXiv:2507.17888*, 2025.



Miles Farmer is an undergraduate computer science and mathematics student at the University of Missouri, where he is a member of the Computational Imaging and Visualization Analysis (CIVA) Laboratory. His current research interests include graph-based deep learning and its applications to software analysis and security, deep learning and optimization techniques, and language processing.



Anne Watson graduated from the University of Missouri with a B.S. in Electrical Engineering and a B.S. in Economics. During her time at the University of Missouri, she was a member of the Computational Imaging and Visualization Analysis (CIVA) Lab. Her research interests include Convolutional Neural Networks and their applications to software vulnerabilities.



Hialo Muniz Carvalho received the B.S degree in Software Engineering from the University of Brasilia, Brazil, in 2014. Currently working as a Guest Researcher at the National Institute of Standards and Technology with the Software Assurance Metrics And Tool Evaluation (SAMATE) group, his current research is focused on developing machine learning models for vulnerability detection in source code. His current research interests include Graph Neural Networks applied to several areas, including software analysis and software security.



Vadim Okun (Member, IEEE) is a Computer Scientist at the National Institute of Standards and Technology, where he is leading the Software Assurance Metrics And Tool Evaluation (SAMATE) team. His current research focuses on software assurance, in particular, the effect of tools on security. Previously, he contributed to the development of automated software testing methods: specification-based mutation testing and combinatorial testing. He received a Ph.D. degree in Computer Science from the University of Maryland Baltimore County.

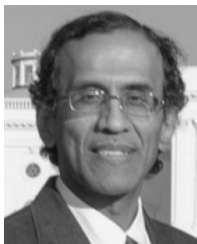


Zineb Maasaoui received the master degree in mathematics and mechanics from ENSEIRB-MATMECA Bordeaux, in France in 2017. Currently a PhD student at the university of Grenoble, France she is also a guest researcher at the National institute of standards and technology with the Software Assurance Metrics And Tool Evaluation group. Her research is focused on datasets evaluation and improvement. Her main domain of interests are AI and software security.



Ekinan Ufuktepe (Member, IEEE) received the B.S. degree in computer engineering from the Izmir University of Economics, in 2011, and the M.S. and Ph.D. degrees in computer engineering from the Izmir Institute of Technology, in 2014 and 2019, respectively. In 2013, he was a Research Intern with the Security and Trust Department, SAP Laboratories France. From 2019 to 2021, he was a Postdoctoral Researcher at the Computational Imaging and Visualization Analysis (CIVA) Laboratory at the University of Missouri-Columbia. He is currently an

Associate Teaching Professor at the University of Missouri-Columbia Electrical Engineering and Computer Science Department. His current research interests include software testing, program analysis, and software security.



Kannappan Palaniappan (Senior Member, IEEE) received the Ph.D. degree from the University of Illinois at Urbana-Champaign, Champaign, IL, USA. He is currently a Curator's Distinguished Professor with the Electrical Engineering and Computer Science Department, University of Missouri. His research interests include the synergistic intersection of image and video processing, computer vision, high-performance computing, and artificial intelligence to understand, quantify, and model physical and computational systems.